一、路由层(2)

二、教学目标

1、完成注册功能

2、完成用户列表功能

3、理解有名和无名的正则表达式

4、理解反向解析前端和后端的使用

三、注册功能

3.1 概述

目前我们已经完成了登录功能 那么注册功能还没完成 所以接下来我们完善功能

3.2 补充代码

`在urls.py`

```
#注册页面
path('register/',views.register)
```

`在views.py`

```
#注册
def register(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        re_password = request.POST.get('re_password')
        # 只要密码和确认密码一致 保存数据到数据库里
        if password == re_password:
            #相当于创建一个 insert into app01_user() ....
            #在django 中相当于新的数据(对象) 返回创建的对象本身
            res =
models.User.objects.create(username=username,password=password)
            print(res,res.username,res.password)
            return  HttpResponse('注册成功')
        else:
            return HttpResponse('两次密码不一致')
```

```
    return render(request,'register.html')
```

`templates` 文件夹中

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
    <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static '/bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
    <!-- 导入外部的js-->
    <script src={% static "/js/jquery-3.7.0.min.js" %}></script>
    <!-- 导入外部的js-->
    <script src={% static "/bootstrap-3.4.1-dist/js/bootstrap.min.js" %}>
</script>

</head>
<body>
<h1 class="text-center">注册</h1>
<div class="row">
    <!--
        一行12份
    左右两边占4份 中间占8份 -->
    <div class="col-md-8 col-md-offset-2">
        <!--
            1. form 表单的 action 参数
            action="" -> 当前页面提交请求
            action="完整url" -> 指定的地址提交请求
            action='/后缀/' -> 本服务器的某个 url 地址提交请求.

            2. method 提交方式
                POST 安全 出现在参数
                GET 不安全 出现在地址栏
        -->
        <form action="" method="post">
            <p>用户名:<input type="text" name="username" class="form-control">
</p>
            <p>密码:<input type="password" name="password" class="form-control">
</p>
            <p>确认密码:<input type="password" name="re_password" class="form-
control"></p>



            <input type="submit" class="btn btn-success btn-block">

        </form>

    </div>
</div>
</body>
</html>
```

完成测试

四、用户列表

4.1 概述

目前登录功能和注册功能都已经实现 但是我们发现用户列表没有在我们的页面中 就无法查询实时的数据列表

4.2 补充代码

`在urls.py`

```
#用户列表
path('userlist/',views.userlist)
```

`在views.py`

```
#用户列表
def userlist(request):
    return render(request, 'userlist.html')
```

`templates` 文件夹中

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
    <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static '/bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
    <!-- 导入外部的js-->
    <script src={% static "/js/jquery-3.7.0.min.js" %}></script>
    <!-- 导入外部的js-->
    <script src={% static "/bootstrap-3.4.1-dist/js/bootstrap.min.js" %}>
</script>

</head>
```

```
<body>
<h1 class="text-center">用户信息</h1>
<div class="row">
    <div class="col-md-8 col-md-offset-2">

        <!--
          Bootstrap 表格基础样式:
          - table           : 基础表格样式（添加边框、内边距等）
          - table-striped   : 斑马条纹效果（交替行背景色，提升可读性）
          - table-hover     : 悬停高亮效果（鼠标悬停时行背景色变化）
        -->
        <table class="table table-striped table-hover">
            <thead>
                <tr>
                    <th>id</th>
                    <th>用户名</th>
                    <th>密码</th>
                </tr>
            </thead>

            <tbody>
            <!-- 目前写死的数据 是不是应该写个循环读取数据? -->
                    <tr>
                        <td>1</td>
                        <td>aaa</td>
                        <td>123</td>
                    </tr>
            </tbody>

        </table>
    </div>
</div>

</body>
</html>
```

完善 在views.py

```
#用户列表
def userlist(request):
    #获取用户表中所有的数据
    #返回的是QuesySet对象
    user_list = models.User.objects.all()
    #<QuerySet [<User: User object (1)>, <User: User object (2)>, <User: User
object (3)>]>
    print(user_list)
    #现在的问题在后端获取到了数据 如何在前端展示?
    #解决方案: 模版函数
    #render 第三个参数
    # 两种写法
    #       1. 以字典的形式传递数据
    #               格式 {'key':value}
    #       2. 以函数的形式传递数据(常用)
    #             locals():将试图函数中所有的数据全部传递给前端页面
    #                       会自动生成也是键值对的形式
    #             区别就是 以字典的形式传递数据可以手动改变键的值
    #                     如果使用locals就不能改变 自动生成键
```

```
    return render(request, 'userlist.html',locals())
```

完善`userlist.html`

```
....
     <tbody>
             <!-- 目前写死的数据 是不是应该写个循环读取数据?
                    <tr>
                        <td>1</td>
                        <td>aaa</td>
                        <td>123</td>
                    </tr>
              -->
             <!--使用模版语法(DTL):
                    for -> 按tab键自动补齐
              -->
                    {% for userli in user_list %}
                        <tr>
                            <td>{{ userli.id }}</td>
                            <td>{{ userli.username }}</td>
                            <td>{{ userli.password }}</td>
                        </tr>
                    {% endfor %}
            </tbody>

....
```

对于for循环的补充:

`for`循环内置了一组可在循环体内使用的变量:

变量	描述
`forloop.counter`	循环计数器，表示从`1`开始的循环索引
`forloop.counter0`	循环计数器，表示从`0`开始的循环索引
`forloop.revcounter`	反向循环计数器，表示从`1`开始的循环索引（即最后一个为`1`）
`forloop.revcounter0`	反向循环计数器，表示从`0`开始的循环索引（即最后一个为`0`）
`forloop.first`	当前循环为首个循环时，该变量为`True`
`forloop.last`	当前循环为最后一个循环时，该变量为`True`
`forloop.parentloop`	在嵌套循环中，指向当前循环的上级循环

```
<tr class="{% if forloop.first %}success{% endif %}">
```

- ### 4.2.1 `if`

`if`标签用于判断给定的变量，其语法结构有三种：

```
{% if expression %}
    ...
{% endif %}


{% if expression %}
    ...
{% else %}
    ...
{% endif %}


{% if expression %}
...
{% elif expression %}
...
{% elif expression %}
...
{% else %}
...
{% endif %}
```

`if` 标签内可以使用运算符 `==`、`!=`、`<`、`>`、`<=`、`>=`、`in`、`not in`、`is` 和 `is not`等。

另外，在`if`标签中也可以使用逻辑运算符，如`not`、`and`、`or`。

- ### 4.2.2 `for` ... `empty`

当传递到`for`标签的列表/字典不存在或者为空时，可以使用`{% empty %}` 标签来指定输出内容。其语法结构是：

```
{% for iterating_variable in sequence %}
    ...
    ...
{% empty %}
    ...
    ...
{% endfor %}
```

防止空值可以添加一下代码

```
<tr class="text-muted">
            <td colspan="3" class="text-center">
                <i class="bi bi-database-exclamation"></i>
                暂无用户数据，请先添加用户
            </td>
</tr>
```

测试页面 此时此刻就能看到从数据库查询到的数据

用户信息

id	用户名	密码	操作
1	daxi	123	编辑 删除
2	zhangsan	234	编辑 删除
3	admin	123	编辑 删除

五、编辑/删除功能

5.1 概述

目前注册和登录功能都已经实现

现在的需求:

在用户信息页面中添加操作列 -> 加 编辑 删除按钮 实现在线编辑和在线删除功能

5.2 有名和无名

前言:

讲一下有名和无名分组的正则表达式

```
- 无名分组:
    - 正则表达式中有个分组的概念,
    - 用()将一段正则表达式括起来..
    - 将正则表达式匹配的内容以位置参数的形式传递给后端的视图函数,
    - 这个就是无名分组
- 有名分组:
      (?P<别名>正则表达式)
    - 将正则表达式匹配的内容以位置参数的形式传递给后端的视图函数,
    - 这个就是无名分组
```

在urls.py 演示无名和有名分组

```
#无名分组
re_path('time/(\d+)/(\d+)',views.time),
#有名分组
re_path('date/(?P<year>\d+)/(?P<month>\d+)',views.date),
```

`在views.py`完善代码

```
#无名正则分组
def time(request,year,month):
    print(year,month)
    return HttpResponse("无名分组")

#有名正则分组
def date(request,year,month):

    print(year, month)
    return HttpResponse("有名分组")
```

注意: 在同一个url里有名分组和无名分组不能混用,要么全部有名 要么全部无名

5.3 反向解析

概念：通过 URL 名称动态生成 URL 路径，避免硬编码。当 URL 路径修改时，无需手动修改所有代码中的链接。

5.3.1 前端反向解析:

案例演示:

`在urls.py`

```
#反向解析的基础演示
path("home/",views.home)
```

`在views.py`

```
def home(request):
    return render(request, 'home.html')
```

`templates中的test.html`

```
<body>
    <h1 class="text-center">我爱python</h1>
    <!-- 写后缀表示 朝当前服务器上的某个路径提交请求
        此时此刻 当我路径没改变的时候都是可以访问的
        假如我把/home路由改了呢?
        比如把path("/homeaaa")改成了这个 是不是意味着下面的路径地址都需要改

        使用反向解析 通过url 关键字 后面跟着 别名
```

```
    -->
    <a href="/home/">前往home页面</a>
    <a href="/home/">前往home页面</a>
    <a href="/home/">前往home页面</a>
    <a href="/home/">前往home页面</a>

</body>
```

发现问题:

无法固定url地址 如果当url路径地址改变了 就会产生前端页面也需要跟着一起修改

解决方案:

反向解析- 通过一些方法得到一个结果,该结果就可以直接访问对应的url去触发视图函数

无论路由匹配的url怎么变化,都要通过方法访问到对应的视图函数

步骤1:

给path添加name参数 意思就是取别名

```
path("home/",views.home,name="home")
```

步骤2:

templates中的test.html

```
<body>
    <h1 class="text-center">我爱python</h1>
    <!-- 写后缀表示 朝当前服务器上的某个路径提交请求
        此时此刻 当我路径没改变的时候都是可以访问的
        假如我把/home路由改了呢?
        比如把path("/homeaaa")改成了这个 是不是意味着下面的路径地址都需要改

        使用反向解析 通过url 关键字 后面跟着 别名
        在前端页面可以通过模版语法指定别名解析到路由,执行对应的视图函数
    -->
    <a href="{% url 'home' %}">前往home页面</a>
    <a href="{% url 'home' %}">前往home页面</a>
    <a href="{% url 'home' %}">前往home页面</a>
    <a href="{% url 'home' %}">前往home页面</a>

</body>
```

- **好处**
  - **解耦性**：URL 路径与代码分离
  - **维护性**：修改 URL 路径时，只需调整 `urls.py`，无需改动模板或视图

5.3.2 后端反向解析:

> 后端想要获取解析的后缀(也就是路径): 通过 reverse

`views.py`

```
from django.shortcuts import render,redirect,reverse
```

接着在home方法中验证

```
def home(request):
    #获取路由中的后缀
    print(reverse('home'))


    return render(request, 'home.html')
```

假如你的是无名分组或者是有名分组 后端反向解析有点复杂 接下来我们演示一下

5.3.2.1 正则无名分组

`urls.py`

```
 #反向解析 无名带正则表达式演示
re_path('retest/(\d+)/', views.retest,name="retest"),
```

`templates中test.html`

```
 <!--当你这样写的时候肯定不行 报错 说没有传递参数
 Reverse for 'retest' with no arguments not found. 1 pattern(s) tried:
['retest/(\\d+)/']
 -->
 {#    <a href="{% url 'retest' %}">我是正则反向解析</a>#}
 <!-- 应该修改成 -->
 <a href="{% url 'retest' 123 %}">我是正则无名反向解析</a>
```

`views.py`

```
#无名正则
def retest(request,test_id):
    print(test_id)
    # 获取路由中的后缀
    # 正确传递参数（使用 args 或 kwargs）
    print(reverse('retest', args=(123 , )))  # 输出 /retest/123/

    return render(request, 'home.html')
```

完成测试

输出

```
123
/retest/123/
```

5.3.2.2 正则有名分组

`urls.py`

```
#反向解析 有名带正则表达式演示
re_path('renametest/(?P<test_id>\d+)/', views.renametest,name="renametest"),
```

`views.py`

```
#有名正则
def renametest(request,test_id):
    print(test_id)
    # 获取路由中的后缀
    # 正确传递参数（使用 args 或 kwargs）
    print(reverse('renametest', args=[test_id]))  # 输出 /renametest/123/
    return render(request, 'home.html')
```

完成测试

输出

```
123
/renametest/123/
```

六、作业

6.1 完成本天的案例